CS6886: Systems for DL: Project Report
Accelerating SAM 3.1 Video Inference Through Temporal Computation Reuse

Aravindh Sriram Kumar CS26S020, Shaik Sadik CS26S010

November 2026

1 Summary of original work

SAM 3.1 detects, segments and tracks every instance of a text-prompted concept across a video [1, 2]. Each frame passes through a 32-block Vision Transformer trunk, a neck, a detector and a memory-based tracker. SAM 2 added the streaming memory [3], SAM 3 added concept prompts, and SAM 3.1 groups tracked objects into buckets. These steps reduce per-object work, but none of them reduces the per-frame encoder cost.

Two earlier works reuse encoder computation between frames. Eventful Transformers [4] gate operations on how much each token's activations changed, recomputing only the top K tokens and reusing cached outputs for the rest. EP-ViT [5] chooses tokens from the motion vectors and residuals of a compressed stream [6] before the encoder runs, so selection costs no GPU work. Neither was built for SAM 3.1. Eventful was evaluated on standalone backbones whose outputs feed a task head directly, and EP-ViT has no released code.

2 Scope for speedup in original work

The tracker reuses information over time, but the trunk does not: it encodes each 1008×1008 frame into 5,184 tokens from scratch and keeps no state between frames. In our compiled baseline the trunk takes 89.9 ms of a 175.0 ms model step (51%). The other 85 ms is untouched by encoder work, so an encoder-only method cannot exceed about 2.1× end to end.

Video leaves much of this work repeated, but unevenly. On our low-motion videos, 87% of pixels move at most 1 px between frames, mean token cosine similarity is 0.86, and 69% of tokens stay above 0.9. On high-motion videos these are 17%, 0.60 and 15%. Yet the dense trunk spends the same time on both, which argues for deciding per token what to recompute. Table 1 shows where the work is. The MLP is the largest and purely per-token target, while attention cannot shrink in the same way, because a selected query still needs the keys and values of its whole span.

Table 1: Cost of one dense trunk pass. Time share is Nsight GPU time of the eager trunk.

Operation	**GMAC**	**MAC share**	**GPU time share**
MLP	1,609.0	59.6%	47.0%
`qkv` projection	521.8	19.3%	13.2%
Output projection (`proj`)	173.9	6.4%	5.2%
Attention (windowed and global)	391.4	14.5%	13.5%
Patch embedding; norms, RoPE, reshapes, casts	3.1	0.1%	21.1%

We ask two questions: how much trunk computation can be removed while keeping the accuracy of the whole pipeline, and how much of that saving becomes measured GPU time?

The second matters because reuse adds selection, gather and scatter work that a dense encoder never does. In SAM 3.1 a reuse error can also persist through the tracker's memory, so accuracy has to be judged on the final masks.

3 Implementation Details

All methods keep SAM 3.1's weights, need no training, and leave everything after the trunk unchanged. The key parts of the implementation are:

1. **experiments/harness/baseline.py:** The baseline, Dense B2d, stores the linear weights in bf16 and compiles the trunk with `torch.compile`. Trunk time falls from 137.0 ms (stock) to 100.5 ms (bf16 fixes) to 92.2 ms (compiled) in an interleaved benchmark, so reuse gets no credit for these free gains.
2. **epvit/codec/encode.py, extract.py:** Each video is encoded with libx264 (CRF 23, an I-frame every 192 frames, I and P frames only, one reference), so every motion vector points into the previous frame. Motion vectors, residuals and intra-coded share are then mapped onto the 72×72 token grid. This runs offline.
3. **epvit/codec/schedule.py:** Each token is classed as static, matchable or unmatchable. A token reused for too long accumulates error, so an update link forces it back when its score exceeds 0.5. The paper recomputes a variable number of tokens. We recompute a fixed 75% per P-frame ($k = 3{,}888$), since compiled graphs need fixed shapes. Figure 1 shows the steps.
4. **epvit/model/epvit.py, install.py:** Selected rows run every block. The others keep or warp cached keys, values and outputs, and selected queries attend to all buffered keys and values. I-frames and the first frame of a session run dense. The encoder replaces the trunk attribute of SAM's neck, so no SAM file changes.

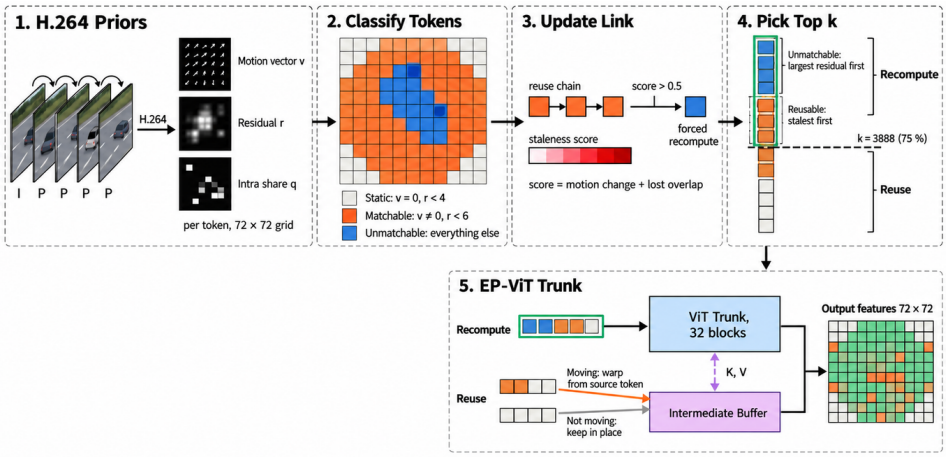


Figure 1: EP-ViT: from H.264 priors to the token plan of one P-frame. The figure is schematic; the real grid is 72×72.

5. **eventful/gates.py:** A gate around an operation keeps a reference holding each token's input when it was last recomputed. Each frame it scores the relative change $\|c_i - \mathrm{ref}_i\| / \|\mathrm{ref}_i\|$, selects the top K tokens, runs the operation on those rows, and scatters the results into a buffer of cached outputs (Figure 2). Each gate adds kernels, so it pays only if it saves more than it costs. Timed alone at K = 75%, only the MLP gate saves time, which is why we initially gated only the MLP. Table 2 shows why that is not the whole story: `qkv` and `proj` lose time at high K but pay at low K. Lagged K produces a mixture of low and high K values across frames and videos, so the frames that gain offset those that lose. Our best configuration, chosen for speed, therefore gates `qkv`, `proj` and the MLP in all 32 blocks. MLP-only stays as the slower, more accurate alternative.

6. **eventful/controller.py:** K adapts to the video. Gates count tokens whose relative change exceeds 0.6, and the next frame's single K, shared by all gates, is the mean count rounded up to a multiple of 512, starting from 3,840. Counts reach the host through a non-blocking copy, so the GPU never stalls, at the cost of K being up to two frames old. When K reaches 4,608 the frame runs dense, which refreshes all caches, and K restarts.
7. **Adaptive selection:** We also explored the proposal's other selection ideas: 3×3 neighbour matching, per-token adaptive thresholds and flow-warped references. In early checks, none gained enough to justify its cost, so we did not adopt them. Lagged K keeps the benefit of adapting to each video without a CPU–GPU sync.
8. **eventful/kernels/:** Top-K is a Triton kernel, references are kept in bf16, and each K bucket has its own compiled graph, replayed with CUDA graphs.
9. **kernels/fused_gemm/:** Gating splits an operation into gather, GEMM and scatter. A fused kernel can remove the copies, but not the GEMM, score or top-K, so its gain is limited by what the copies cost. We built one CUTLASS [9] kernel that reads and writes rows by index, and targeted the MLP gate, which gives most of the saving and is common to every Eventful configuration.

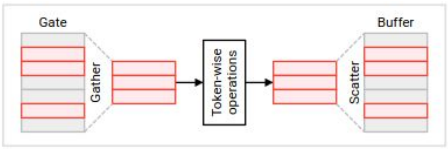


Figure 2: Eventful: selected tokens are gathered, processed and scattered back into a buffer of cached outputs. Reproduced from Dutson et al. [4].

Table 2: Net trunk time saved per gate with all three gates on, in Nsight ms at one clock (negative is slower). The last two columns weight each K by how often it occurred in that subset's run.

Gate	**K=512**	**1,536**	**2,048**	**3,584**	**3,840**	`normal`	`high_motion`
MLP	38.6	31.4	29.3	14.0	14.6	32.6	11.9
`qkv`	9.2	4.7	4.3	−2.0	−2.3	5.6	−0.2
`proj`	2.8	1.7	1.4	−1.2	−1.3	1.9	−0.3

Evaluation setup. We use the SA-V validation split of SA-Co/VEval [8]: the 10 lowest-motion (`normal`) and 10 highest-motion (`high_motion`) videos, fixed before any reuse experiment. They bracket the benchmark rather than sample it, so we report them separately. All methods run at the native 24 fps. We evaluate 189 (video, phrase) pairs. pHOTA [7] is our primary measure, and cgF1 [1] is also reported. Encoder time is trunk GPU time per frame, including selection, gather, scatter and host planning. End-to-end (E2E) time is the median model step.

4 Results

Table 3 compares each method with the dense baseline. EP-ViT is steady at about 1.32× because its budget is fixed. Eventful adapts: three gates recomputes 27% of rows on `normal` but 75% on `high_motion`. It matches EP-ViT overall and is the fastest method on `normal`, but has the lowest accuracy in every column of Tables 3 and 4. MLP-only recomputes 31% and 86%. The shared K explains part of the gain: mean K falls from 1,618 to 1,383 on `normal` and from 4,463 to 3,892 on `high_motion`, where dense fallback frames drop from 61% to 31%. On `normal`, Table 2 credits `qkv` and `proj` with roughly 7.5 ms of the 10.8 ms saved over MLP-only.

Table 3: Results on the 189 scored pairs. Accuracy is ×100; encoder (Enc.) time is trunk ms per frame; speedups are against dense. Eventful MLP gates the MLP only.

Method	**cgF1**	**pHOTA**	**ΔpHOTA**	**Enc. ms**	**Speedup**	`normal`	`high_motion`
Dense B2d (baseline)	12.41	48.88	0	89.9	1.00×	1.00×	1.00×
EP-ViT 75%	12.26	48.27	−0.61	68.3	**1.32**×	1.31×	1.32×
Eventful 3 gates	11.25	47.48	−1.40	68.7	1.31×	**1.90**×	1.11×
Eventful MLP	12.27	**48.97**	+0.09	75.5	1.19×	1.55×	1.05×
Eventful MLP + fused	11.55	48.57	−0.32	75.2	1.20×	1.53×	1.06×

Table 4: The same methods by video subset.

	`normal` (70 pairs)			`high_motion` (119 pairs)		
Method	**cgF1**	**pHOTA**	**Encoder ms**	**cgF1**	**pHOTA**	**Encoder ms**
Dense B2d (baseline)	15.43	55.45	89.4	11.00	38.35	90.2
EP-ViT 75%	14.63	54.25	68.5	11.11	38.88	68.2
Eventful 3 gates	14.30	53.53	47.0	9.94	37.34	81.4
Eventful MLP	15.61	55.38	57.8	10.78	38.61	85.9
Eventful MLP + fused	14.78	55.35	58.4	9.96	37.80	85.0

Table 5: EP-ViT settings (90%, 80% and 60% omitted). Speedup is E2E against dense in the same round. Bold: chosen setting.

	pHOTA		**E2E speedup**	
Budget (operations gated)	`normal`	`high_motion`	`normal`	`high_motion`
Dense B2d	55.45	38.35	1.00×	1.00×
100% (all)	55.37	38.34	0.97×	0.97×
75% (all)	**54.25**	**38.88**	**1.15**×	**1.13**×
50% (all)	53.83	36.28	1.27×	1.25×
75% (`qkv` only)	54.55	38.64	1.01×	1.00×
75% (MLP only)	53.41	38.76	1.08×	1.07×

On `high_motion` they save nothing, so the 4.5 ms gain there comes from the lower K. The fused kernel gives no useful gain.

Table 5 gives the EP-ViT settings behind the 75% choice. A 100% budget matches dense pHOTA but is 3% slower end to end, the fixed price of its attention path. Lower budgets are faster but lose pHOTA on `normal`. Gating fewer operations does not pay.

Table 6 gives the median model step from the full 189-pair sweeps.

Fused kernel. It saves only about 0.2 ms per frame in the trunk benchmark (74.7 to 74.5 ms). At K = 3,840 it removes the 0.81 ms scatter and 0.30 ms of gather, but reading rows by index slows `fc1` by about 10% (+1.11 ms), so kernel time ends level. It does not beat MLP-only.

Accuracy. Eventful MLP stays closest to dense: overall pHOTA is 48.97 against 48.88, and cgF1 is 12.27 against 12.41. EP-ViT loses a little, 0.61 pHOTA overall, all of it on `normal`; on `high_motion` it is slightly above dense. Three gates gives up the most accuracy for its speed, with overall pHOTA down 1.40 and cgF1 down from 12.41 to 11.25, and the largest loss on `normal` (1.92 pHOTA). The fused kernel holds pHOTA close to dense on `normal` (55.35 against 55.45), but cgF1 falls to 11.55 overall. Every method, including dense, scores far lower on `high_motion` (pHOTA about 38 against 55), so that gap comes from the harder videos, not from reuse.

Choosing a method. EP-ViT suits steady speedup, but it needs a stream that already carries motion vectors: building priors offline costs about 162 ms per frame on our CPU, several

Table 6: End-to-end model step on the 189 pairs, in ms.

Method	Dense	Method	E2E speedup
EP-ViT 75%	175.0	137.8	1.27×
Eventful 3 gates	175.0	144.2	1.21×
Eventful MLP	175.0	158.1	1.11×
Eventful MLP + fused	175.0	155.2	1.13×

times what it saves. Eventful needs no codec and suits low-motion video, but gains little on busy video. Three gates is the choice when speed matters most, and MLP-only when accuracy must stay close to dense. Temporal reuse gives real but bounded gains, and a larger gain needs reuse beyond the trunk.

Limitations. All the methods change only the trunk, so they cannot exceed about 2.1× end to end, however much they reuse. EP-ViT's budget is fixed by compiled graph shapes, so it gains no more on static video than on busy video, and it depends on motion data from a compressed stream. Eventful's gain falls as motion rises, its K can be two frames old, so it reacts late to sudden changes, and its gates cost time: at high K, `qkv` and `proj` are slower than no gate at all. In both methods a reuse error can persist through the tracker's memory, so it shows only on the final masks.

5 Source code

All the source code is available at `https://github.com/AravindhSriramKumar/sam31-video-speedup`, with the evaluation pairs, a sample video, saved results and instructions to reproduce them.

References

[1] N. Carion et al., "SAM 3: Segment Anything with Concepts", ICLR 2026, arXiv:2511.16719. Code: `https://github.com/facebookresearch/sam3`

[2] Meta Superintelligence Labs, "SAM 3.1 Release Notes: Object Multiplex", 2026. `https://github.com/facebookresearch/sam3/blob/main/RELEASE_SAM3p1.md`

[3] N. Ravi et al., "SAM 2: Segment Anything in Images and Videos", ICLR 2025. Code: `https://github.com/facebookresearch/sam2`

[4] M. Dutson, Y. Li and M. Gupta, "Eventful Transformers: Leveraging Temporal Redundancy in Vision Transformers", ICCV 2023. Code: `https://github.com/WISION-Lab/eventful-transformer`

[5] Y. Qiu, Q. Wang, Z. Li and Q. Liu, "EP-ViT: Leveraging Encoding Priors to Eliminate Spatio-Temporal Redundancy in Vision Transformer for Efficient Inference", ECAI 2025.

[6] T. Wiegand et al., "Overview of the H.264/AVC Video Coding Standard", IEEE TCSVT 13(7), 2003.

[7] J. Luiten et al., "HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking", IJCV 129, 2021.

[8] SA-Co/VEval dataset: `https://huggingface.co/datasets/facebook/SACo-VEval`. SA-V frames: `https://sa-co.roboflow.com/veval/saco_sav.zip`

[9] NVIDIA CUTLASS, v3.8.0: `https://github.com/NVIDIA/cutlass`